Python meetup

Plaidoyer pour monde sans ORM

Qui suis je?

Jonathan Leger

Développeur Python / SRE / DevOps...

 Scaleway – Facturation*

* : important pour la suite

Disclaimer

En vrai, les ORM c’est pas si mal…

Fallait bien vous faire venir

Mon parcours

Code:
Lignes de code+++, complexe* et pas forcément totalement maîtrisé

Étapes:
Spécifications → Développement → Test → Déploiement → Astreinte*

* : important pour la suite


POSTGRES
SQL
ORM

Cas d’usage / mon expérience

ASTREINTE:

Samedi Soir
Ou Dimanche Matin...

Mise en prod d’un code que vous ne connaissez pas

Indice: le code est terriblement LENT...

Cas d’usage / mon expérience

→ Code ORM complexe

→ Requête construite à la volée / dépendante de l’entrée

→ Comment voire le vrai code exécuté en production?

Sera t’il lisible / compréhensible?

Cas d’usage / mon expérience

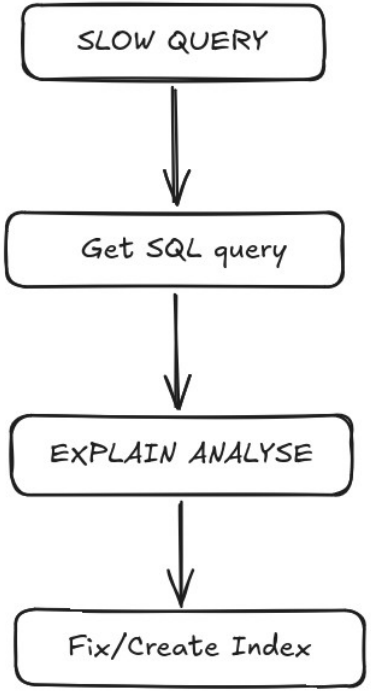
Coupable: **lazy_join**

Marche en local, recette

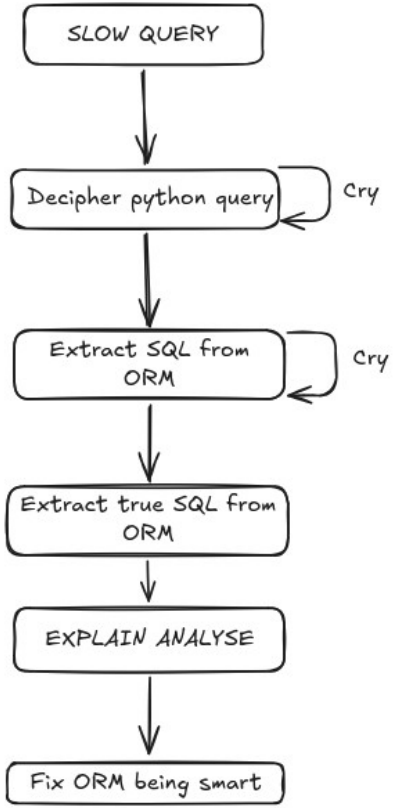
Explose en prod

Magie de SqlAchemy, pas coupable mais pas innocent...

Cas SQL



Cas ORM


SLOW QUERY
Decipher python query
Cry
Extract SQL from ORM
Cry
Extract true SQL from ORM
EXPLAIN ANALYSE
Fix ORM being smart

Cas d’usage / mon expérience

Double peine:

- Maîtriser SQL
- Maîtriser ORM

Le code ORM n’est pas le code SQL

Le SQL généré n’est pas simplement lisible ni accessible

ORM != Principe de moindre surprise

Migration hors de l’ORM → réécriture complète / longue

Bonus

Article:

Learn SQL Once, Use It for 30 Years

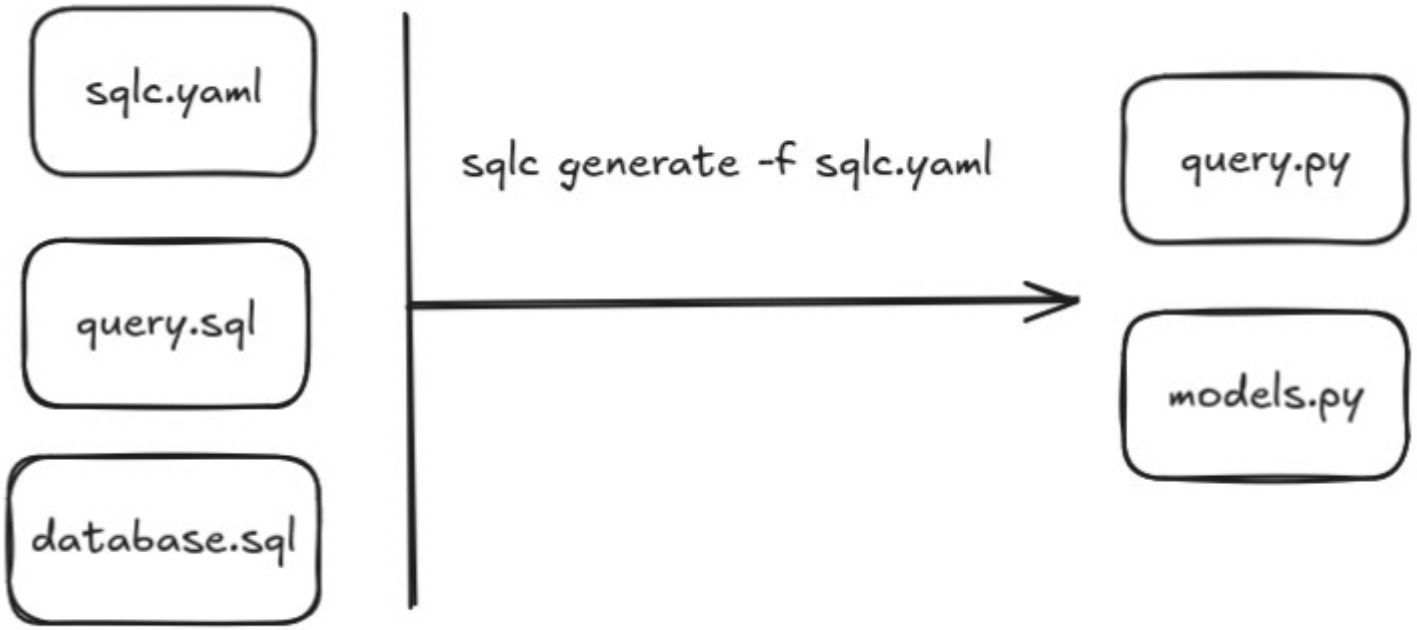
https://fagnerbrack.com/learn-sql-once-use-it-for-30-years-9aceb0bdee03

Solutions logiciels

- SQLC
- IA?
- A la main?

SqlAlchemy est toujours utilisé (un peu)

Sqlc


sqlc.yaml
query.sql
database.sql
sqlc generate -f sqlc.yaml
query.py
models.py

Entrée

database.sql

```sql
CREATE TABLE IF NOT EXISTS author (
    id      INTEGER PRIMARY KEY,
    name    TEXT NOT NULL,
    country TEXT NOT NULL
);
```

query.sql

```sql
-- name: CreateAuthor :one
INSERT INTO public.author (
id, name, country)
VALUES (
  @id::integer,
  @name::text,
  @country::text
)
RETURNING id, name, country
;
```

Retour → :one :many :exec

Sortie: models.py

```python
import dataclasses


@dataclasses.dataclass()
class Author:
    id: int
    name: str
    country: str
```

Sortie: query.py (1)

```
CREATE_AUTHOR = """-- name: create_author \\:one
INSERT INTO public.author (id, name, country)
VALUES (
  :p1\\:\\:integer,
  :p2\\:\\:text,
  :p3\\:\\:text
)
RETURNING id, name, country
"""
```

Sortie: query.py (2)

```python
class Querier:  2+ usages
    def __init__(self, conn: sqlalchemy.engine.Connection):
        self._conn = conn

    def create_author(self, *, id: int, name: str, country: str) -> Optional[models.Author]:
        row = self._conn.execute(sqlalchemy.text(CREATE_AUTHOR),
                                 parameters: {"p1": id, "p2": name, "p3": country}).first()
        if row is None:
            return None
        return models.Author(
            id=row[0],
            name=row[1],
            country=row[2],
        )
```

Utilisation:

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import Session
from authors.query import Querier

def main():  1 usage
    engine = create_engine(DB_URL, echo=False)
    with Session(engine) as session:
        author_querier = Querier(session)
        author_querier.create_author(id=1, name="Sagan", country="France")
        session.commit()
```

Sqlc: exemple avancé

```sql
-- name: ListAuthor :many
SELECT
    id,
    name,
    country
FROM public.author
WHERE
    (sqlc.narg(author_ids)::integer [] IS NULL OR id = any(sqlc.narg(author_ids)::integer []))
    AND (sqlc.narg(author_names)::text [] IS NULL OR name = any(sqlc.narg(author_names)::text []))
    AND (sqlc.narg(author_countries)::text [] IS NULL OR country = any(sqlc.narg(author_countries)::text []))
ORDER BY
    CASE WHEN sqlc.narg(order_by)::text ILIKE '%author_name_asc%' THEN name END ASC,
    CASE WHEN sqlc.narg(order_by)::text ILIKE '%author_name_desc%' THEN name END DESC,
    CASE WHEN sqlc.narg(order_by)::text ILIKE '%author_countries_asc%' THEN country END ASC,
    CASE WHEN sqlc.narg(order_by)::text ILIKE '%author_countries_desc%' THEN country END ASC
LIMIT sqlc.narg(page_size) OFFSET sqlc.narg(page_offset)
;
```

Sqlc: exemple avancé

```python
@dataclasses.dataclass()  1 usage
class ListAuthorParams:
    author_ids: Optional[List[int]]
    author_names: Optional[List[str]]
    name: Optional[List[str]]
    author_countries: Optional[List[str]]
    order_by: Optional[str]
    page_offset: Optional[int]
    page_size: Optional[int]
```

Avantages

- SQL peu modifié*
- Typage automatique
- Génération du code intermédiaire

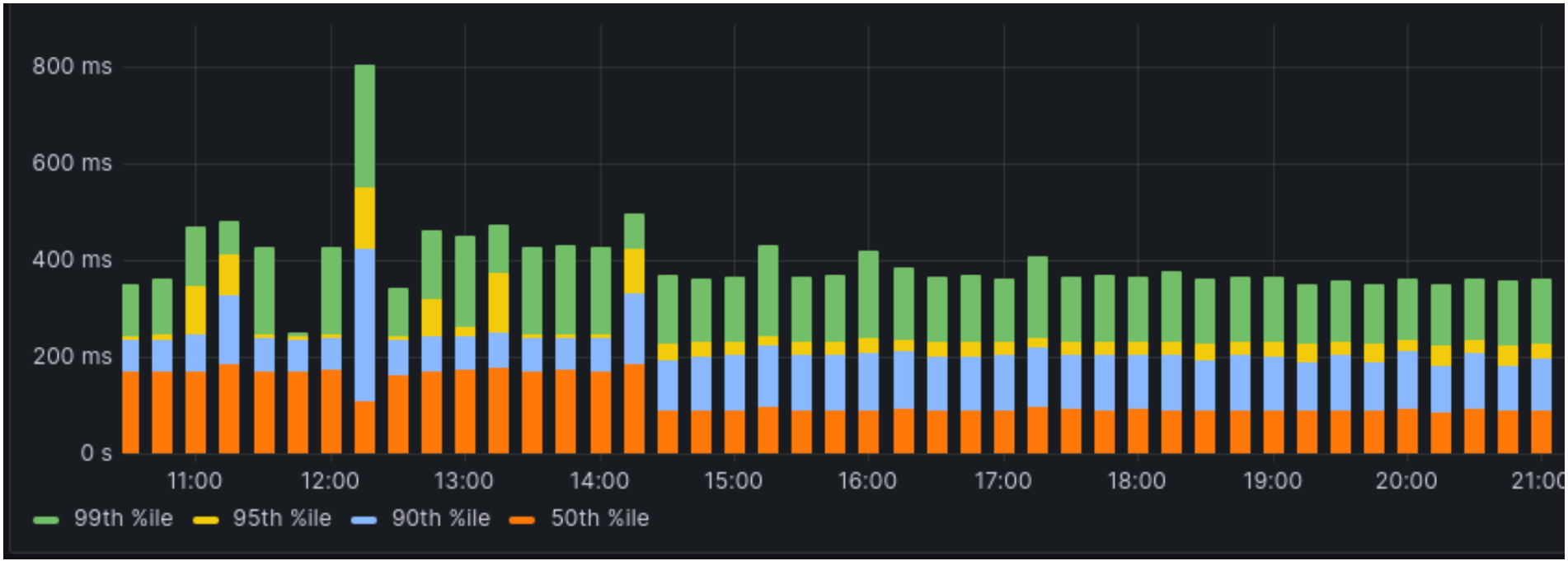
Inconvénients

- SQL modifié quand même
- Compilation nécessaire à chaque modification
- Librairie en plus
- SQLC reste une librairie GO avec option Python

En résumé

- ORM != SQL
- Maîtriser le SQL c’est mieux

Bonus: performance API



Anecdote Bis

Script de générations de rapports:

- Pandas: puissant mais complexe
- Plus de maîtrise dans l’équipe
- Exécution en 40min
- Réécriture Python basique < 1Min

FIN

- Première présentation Meetup
- Ceci est mon opinion
- Retours? Améliorations?

MERCI

QUESTIONS ?